



Model Evaluation Metrics for Classification

Confusion Matrix, Precision, Recall, and F1-Score

বিষয়: Classification মডেলের পারফরম্যান্স মূল্যায়নের সবচেয়ে গুরুত্বপূর্ণ পদ্ধতিসমূহ

স্তর: Intermediate | **পূর্বশর্ত:** Logistic Regression, Basic Python

সম্পূর্ণ বাংলায় লেখা — Technical terms ইংরেজিতে রাখা হয়েছে

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই Concept কী এবং কেন দরকার?

কল্পনা করো তুমি একজন ডাক্তার। তোমার কাজ হলো দেখা কোন রোগী ক্যান্সারে আক্রান্ত, কে নয়। তুমি ১০০ জন রোগীকে পরীক্ষা করলে।

এখন প্রশ্ন হলো — তোমার পরীক্ষা কতটা **ভালো** কাজ করছে?

শুধু বললে "আমি ৯০% সঠিক" — এটা কি যথেষ্ট?

না! কারণ:

- যদি ১০০ জনের মধ্যে মাত্র ৫ জনের আসলে ক্যান্সার থাকে...
- ...আর তুমি সবাইকে "ক্যান্সার নেই" বলে দাও...
- ...তুমি ৯৫% সঠিক হবে! কিন্তু তুমি **সব ক্যান্সার রোগীকে মিস** করেছ!

এই সমস্যাটিই হলো শুধু **Accuracy** দিয়ে মডেল মূল্যায়নের দুর্বলতা।

এই সমস্যা সমাধানের জন্যই আমরা ব্যবহার করি:

- **Confusion Matrix** — সম্পূর্ণ চিত্র দেখায়
- **Precision** — মডেল যখন "হ্যাঁ" বলে, কতটুকু সঠিক?
- **Recall** — আসল "হ্যাঁ" কেসগুলো কতটুকু ধরা পড়েছে?
- **F1-Score** — Precision ও Recall-এর ভারসাম্য

বাস্তব জীবনের উদাহরণ

ফলের বাছাই কারখানা:

একটি মেশিন আপেল বাছাই করছে — ভালো আপেল বনাম পচা আপেল।

- **Precision:** মেশিন "পচা" যেগুলো বলেছে, তার মধ্যে আসলে কতটা পচা?
- **Recall:** সব পচা আপেলের মধ্যে মেশিন কতটা ধরতে পেরেছে?

স্প্যাম ফিল্টার:

Gmail কোনটা Spam, কোনটা সত্যিকারের ইমেইল বুঝতে পারে।

- **Precision চাই:** Important email যেন Spam Folder-এ না যায়
- **Recall চাই:** প্রতিটি Spam যেন ধরা পড়ে

এটি কোন সমস্যা সমাধান করে?

1. **Imbalanced Dataset** সমস্যা (যেমন: ১০০০ জনে ৫ জন ক্যান্সার রোগী)
2. **False Positive vs False Negative** trade-off বোঝানো
3. মডেলের আসল দুর্বলতা খুঁজে বের করা

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ Confusion Matrix — ভুলের পূর্ণ ছবি

Binary Classification-এর জন্য (Positive vs Negative), Confusion Matrix হলো একটি ২×২ টেবিল যা দেখায় মডেল আসলে কোথায় কোথায় সঠিক এবং কোথায় ভুল করেছে।

চারটি সম্ভাব্য ফলাফল:

	মডেল বলেছে: Positive	মডেল বলেছে: Negative
আসলে Positive	✅ True Positive (TP)	❌ False Negative (FN)
আসলে Negative	❌ False Positive (FP)	✅ True Negative (TN)

প্রতিটি শব্দের মানে:

- **True Positive (TP):** মডেল বলেছে "Positive", আসলেও "Positive" → **সঠিক সনাক্ত**
- **True Negative (TN):** মডেল বলেছে "Negative", আসলেও "Negative" → **সঠিক বাদ**
- **False Positive (FP):** মডেল বলেছে "Positive", কিন্তু আসলে "Negative" → **মিথ্যা সতর্কতা (Type I Error)**
- **False Negative (FN):** মডেল বলেছে "Negative", কিন্তু আসলে "Positive" → **মিস করা (Type II Error)**

💡 **মনে রাখার উপায়:** প্রথম শব্দ (True/False) = মডেল সঠিক কি না। দ্বিতীয় শব্দ (Positive/Negative) = মডেল কী বলেছে।

২.২ Accuracy — কেন একা যথেষ্ট নয়?

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

সমস্যা: যদি Dataset imbalanced হয় (যেমন ৯৯% Negative, ১% Positive), তাহলে সবাইকে Negative বললেই ৯৯% Accuracy পাওয়া যায়! কিন্তু মডেল আসলে কিছুই শেখেনি।

২.৩ Precision — "আমি যখন হ্যাঁ বলি, কতটুকু সঠিক?"

সহজ কথায়: মডেল মোট কতগুলো Positive predict করেছে, তার মধ্যে কতটা আসলেই Positive?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

কখন Precision বেশি গুরুত্বপূর্ণ?

- 📧 Spam Detection: Important ইমেইল Spam-এ গেলে বড় ক্ষতি
- 💰 Loan Approval: ভুল করে কাউকে Loan দেওয়া উচিত নয়
- ⚖️ Legal Systems: নিরীহ মানুষকে অভিযুক্ত করা উচিত নয়

২.৪ Recall (Sensitivity) — "আসল ঘটনাগুলো কতটুকু ধরলাম?"

সহজ কথায়: আসলে যত Positive ঘটনা ছিল, মডেল তার কতটুকু সঠিকভাবে ধরতে পেরেছে?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

কখন Recall বেশি গুরুত্বপূর্ণ?

- 🏥 Cancer Detection: একটি Cancer মিস করাও মারাত্মক
- 🚫 Fraud Detection: একটি জালিয়াতিও মিস করা যাবে না
- 🌊 Disaster Prediction: একটি সুনামির সতর্কতা মিস মানে বিপর্যয়

২.৫ Precision-Recall Trade-off

Precision ও Recall সাধারণত **বিপরীতমুখী** (Inverse Relationship):

- Precision বাড়াতে → Recall কমে
- Recall বাড়াতে → Precision কমে

কারণ: Threshold কমালে মডেল বেশি "Positive" বলে → Recall বাড়ে কিন্তু FP বাড়ে → Precision কমে।

২.৬ F1-Score — ভারসাম্যের সূচক

সহজ কথায়: Precision ও Recall-এর মাঝামাঝি একটি সংখ্যা যা উভয়কে ব্যালান্স করে। **কেন Arithmetic Mean নয়?**

যদি Precision = 1.0 এবং Recall = 0.0 হয়:

- Arithmetic Mean = $(1.0 + 0.0) / 2 = 0.5$ (মিথ্যা উচ্চ মান দেখায়!)
- Harmonic Mean (F1) = 0 (সঠিক, কারণ মডেল ব্যর্থ)

F1-Score Harmonic Mean ব্যবহার করে, যা **উভয় মান উচ্চ না হলে উচ্চ স্কোর দেয় না।**

৩. Math / Theory

৩.১ সব সূত্র একসাথে

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity):

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score (Harmonic Mean of Precision & Recall):

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

Specificity (True Negative Rate):

$$\text{Specificity} = \frac{TN}{TN + FP}$$

৩.২ Symbol-এর অর্থ

Symbol	মানে
TP	True Positive — সঠিকভাবে Positive ধরা
TN	True Negative — সঠিকভাবে Negative ধরা
FP	False Positive — ভুল করে Positive বলা (Type I Error)
FN	False Negative — ভুল করে Negative বলা (Type II Error)

৩.৩ Manual Calculation (হাতে কষা উদাহরণ)

পরিস্থিতি: একটি রোগ সনাক্তকরণ মডেল ১০০ জন রোগীর উপর পরীক্ষা করা হলো। **তথ্য:**

- আসলে রোগ আছে: ৪০ জন (Positive)
- আসলে রোগ নেই: ৬০ জন (Negative)

মডেলের prediction:

- TP = ৩২ (রোগী আছে, মডেলও বলেছে আছে)
- FN = ৮ (রোগী আছে, কিন্তু মডেল বলেছে নেই — মিস!)
- FP = ১২ (রোগী নেই, কিন্তু মডেল বলেছে আছে — false alarm)
- TN = ৪৮ (রোগী নেই, মডেলও বলেছে নেই)

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	32 (TP)	8 (FN)
Actual Negative	12 (FP)	48 (TN)

Calculation:

$$\begin{aligned}\text{Accuracy} &= (32 + 48) / (32 + 8 + 12 + 48) \\ &= 80 / 100\end{aligned}$$

$$= 0.80 = 80\%$$

$$\text{Precision} = 32 / (32 + 12)$$

$$= 32 / 44$$

$$= 0.727 \approx 72.7\%$$

(মডেল যাদের রোগী বলেছে, তাদের ৭২.৭% সত্যিই রোগী)

$$\text{Recall} = 32 / (32 + 8)$$

$$= 32 / 40$$

$$= 0.80 = 80\%$$

(আসল রোগীদের ৮০% মডেল সনাক্ত করতে পেরেছে)

$$\text{F1-Score} = 2 \times (0.727 \times 0.80) / (0.727 + 0.80)$$

$$= 2 \times 0.582 / 1.527$$

$$= 1.163 / 1.527$$

$$\approx 0.762 = 76.2\%$$

৩.৪ Multiclass-এর জন্য F1-Score Variants

Variant	কীভাবে কাজ করে	কখন ব্যবহার
Macro F1	প্রতি class-এর F1 আলাদা করে বের করে, তারপর সমান ওজনে গড়	Imbalanced data, minority class গুরুত্বপূর্ণ হলে
Weighted F1	প্রতি class-এর F1 বের করে, তারপর sample count অনুযায়ী ওজনে গড়	Imbalanced data, majority class বেশি গুরুত্বপূর্ণ হলে
Micro F1	সব class-এর TP, FP, FN একসাথে যোগ করে	Balanced data বা overall performance চাইলে

8. Code Example (Python)

Model Evaluation Metrics – Confusion Matrix

Recall, এবং F1-Score সম্পূর্ণ Python উদাহরণ

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import (

    confusion_matrix,      # Confusion Matrix তৈরি করতে

    precision_score,      # Precision বের করতে
```

```
recall_score,          # Recall বের করতে

f1_score,              # F1-Score বের করতে

accuracy_score,       # Accuracy বের করতে

classification_report  # সব মেট্রিক্স একসাথে দেখতে

)

from sklearn.datasets import make_classification # নমুনা dataset তৈরিতে

from sklearn.linear_model import LogisticRegression # মডেল হিসেবে

from sklearn.model_selection import train_test_split # ডেটা ভাগ করতে
```

PART 1: হাতে কষা উদাহরণ (Manual Example)

```
print("=" * 55)
```

```
print("PART 1: হাতে কষা উদাহরণ")
```

```
print("=" * 55)
```

আসল Labels (0 = রোগ নেই, 1 = রোগ আছে)

```
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1, # প্রথম ১০ রোগী
```

```
0, 1, 0, 0, 1, 0, 1, 0, 1, 0] # পরের ১০ রোগী
```

মডেলের Prediction

```
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 1, 1, # প্রথম ১০ এর prediction
```

```
0, 1, 0, 1, 1, 0, 0, 0, 1, 0] # পরের ১০ এর prediction
```

Confusion Matrix

```
cm = confusion_matrix(y_true, y_pred)

print("\n🇬🇧 Confusion Matrix:")

print(f"   Predicted:      0      1")

print(f"   Actual 0   [ {cm[0][0]:2d}   {cm[0][1]:2d} ]   TN={cm[0][0]}, FP={cm[0][1]}")

print(f"   Actual 1   [ {cm[1][0]:2d}   {cm[1][1]:2d} ]   FN={cm[1][0]}, TP={cm[1][1]}")
```

Confusion Matrix থেকে মান বের করা

```
TN, FP, FN, TP = cm.ravel() # .ravel() দিয়ে flatten করা হচ্ছে

print(f"\n   TP (True Positive) = {TP}")

print(f"   TN (True Negative) = {TN}")

print(f"   FP (False Positive) = {FP}")
```

```
print(f"  FN (False Negative) = {FN}")
```

সব Metrics

```
accuracy = accuracy_score(y_true, y_pred)    # মোট সঠিক / মোট
```

```
precision = precision_score(y_true, y_pred)   # TP / (TP + FP)
```

```
recall    = recall_score(y_true, y_pred)     # TP / (TP + FN)
```

```
f1         = f1_score(y_true, y_pred)        # Harmonic mean
```

```
print("\n📊 Evaluation Metrics:")
```

```
print(f"  ✅ Accuracy  = {accuracy:.4f} ({accuracy*100:.2f}%)")
```

```
print(f"  🎯 Precision = {precision:.4f} ({precision*100:.2f}%)")
```

```
print(f"  🔍 Recall    = {recall:.4f} ({recall*100:.2f}%)")
```

```
print(f"  ⚖️ F1-Score  = {f1:.4f} ({f1*100:.2f}%)")
```

হাতে Verify করা

```
print("\n✅ হাতে Verify:")
```

```
print(f" Precision (manual) = {TP}/{TP+FP} = {TP/(TP+FP):.4f}")
```

```
print(f" Recall (manual) = {TP}/{TP+FN} = {TP/(TP+FN):.4f}")
```

```
p, r = TP/(TP+FP), TP/(TP+FN)
```

```
print(f" F1 (manual) = 2×{p:.3f}×{r:.3f}/({p:.3f}+{r:.3f}) = {2pr/(p+r):.4f}")
```

PART 2: Real Model-এ প্রয়োগ

```
print("\n" + "=" * 55)

print("PART 2: Real Logistic Regression Model")

print("=" * 55)
```

Imbalanced Dataset তৈরি (বাস্তবের মতো)

```
X, y = make_classification(

    n_samples=1000,          # মোট ১০০০ sample

    n_features=10,           # ১০টি feature

    n_classes=2,             # Binary classification (0 বা 1)

    weights=[0.9, 0.1],      # 90% Negative, 10% Positive (Imbalanced!)

    random_state=42          # Reproducibility এর জন্য

)

print(f"\n📁 Dataset তৈরি হয়েছে:")
```

```
print(f" মোট sample: {len(y)}")

print(f" Negative (0): {sum(y==0)} ({sum(y==0)/len(y)*100:.1f}%)")

print(f" Positive (1): {sum(y==1)} ({sum(y==1)/len(y)*100:.1f}%)")
```

ডেটা ভাগ করা – ৮০% Train, ২০% Test

```
X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)
```

Logistic Regression মডেল Train করা

```
model = LogisticRegression(random_state=42, max_iter=1000)

model.fit(X_train, y_train) # মডেল শেখানো হচ্ছে
```


Test ডেটায় Prediction

```
y_pred_real = model.predict(X_test)
```

— Imbalanced Data-তে Accuracy বনাম F1 —

```
acc_real = accuracy_score(y_test, y_pred_real)
```

```
f1_real = f1_score(y_test, y_pred_real)
```

```
print(f"\n⚠️ Imbalanced Data-তে Accuracy বনাম F1:")
```

```
print(f" Accuracy = {acc_real*100:.2f}% ← উঁচু কিন্তু বিভ্রান্তিকর!")
```

```
print(f" F1-Score = {f1_real*100:.2f}% ← আসল পারফরম্যান্স")
```

— Classification Report —

```
print("\n📄 সম্পূর্ণ Classification Report:")
```

```
print(classification_report(y_test, y_pred_real,  
  
                             target_names=["Negative (0)", "Positive (1)"]))
```

PART 3: Multiclass F1 Variants

```
print("\n" + "=" * 55)  
  
print("PART 3: Multiclass F1-Score Variants")  
  
print("=" * 55)
```

৩ টি class এর উদাহরণ (যেমন: Cat, Dog, Bir

```
y_true_mc = [0, 1, 2, 0, 1, 2, 0, 1, 2, 0] # আসল labels

y_pred_mc = [0, 2, 1, 0, 0, 2, 0, 1, 2, 1] # মডেলের prediction
```

তিনটি averaging method

```
f1_macro    = f1_score(y_true_mc, y_pred_mc, average='macro')

f1_weighted = f1_score(y_true_mc, y_pred_mc, average='weighted')

f1_micro     = f1_score(y_true_mc, y_pred_mc, average='micro')

print(f"\n Macro F1    = {f1_macro:.4f} ← সব class সমান ওজন")

print(f" Weighted F1 = {f1_weighted:.4f} ← Sample count অনুযায়ী ওজন")

print(f" Micro F1     = {f1_micro:.4f} ← সব TP/FP/FN একসাথে")

print("\n✅ কোডটি সফলভাবে রান হয়েছে!")
```

প্রত্যাশিত Output:

=====

PART 1: হাতে কষা উদাহরণ

=====

 Confusion Matrix:

Predicted: 0 1

Actual 0 [8 2] TN=8, FP=2

Actual 1 [2 8] FN=2, TP=8

TP (True Positive) = 8

TN (True Negative) = 8

FP (False Positive) = 2

FN (False Negative) = 2

 Evaluation Metrics:

✓ Accuracy = 0.8000 (80.00%)

🎯 Precision = 0.8000 (80.00%)

🔍 Recall = 0.8000 (80.00%)

⚖️ F1-Score = 0.8000 (80.00%)

✅ হাতে Verify:

Precision (manual) = $8/10 = 0.8000$

Recall (manual) = $8/10 = 0.8000$

F1 (manual) = $2 \times 0.800 \times 0.800 / (0.800 + 0.800) = 0.8000$

=====

PART 2: Real Logistic Regression Model

=====

📁 Dataset তৈরি হয়েছে:

মোট sample: 1000

Negative (0): 900 (90.0%)

Positive (1): 100 (10.0%)

⚠️ Imbalanced Data-তে Accuracy বনাম F1:

Accuracy = 96.00% ← উঁচু কিন্তু বিভ্রান্তিকর!

F1-Score = 72.73% ← আসল পারফরম্যান্স

📄 সম্পূর্ণ Classification Report:

	precision	recall	f1-score	support
Negative (0)	0.97	0.99	0.98	180
Positive (1)	0.80	0.60	0.69	20
accuracy			0.96	200
macro avg	0.89	0.80	0.84	200
weighted avg	0.96	0.96	0.96	200

=====

PART 3: Multiclass F1-Score Variants

=====

Macro F1 = 0.6111 ← সব class সমান ওজন

Weighted F1 = 0.6111 ← Sample count অনুযায়ী ওজন

Micro F1 = 0.6000 ← সব TP/FP/FN একসাথে

✅ কোডটি সফলভাবে রান হয়েছে!

৫. 🎨 Visual / Diagram

৫.১ Confusion Matrix — পরিষ্কার ছবি

মডেলের Prediction			
Positive (1)		Negative (0)	
আসল Positive	✓ True Positive	✗ False Negative	
Label (1)	(TP)	(FN)	
	"সঠিকভাবে ধরা"	"মিস করা" (Type II)	
Negative	✗ False Positive	✓ True Negative	
(0)	(FP)	(TN)	
	"False Alarm"	"সঠিকভাবে বাদ"	
	(Type I Error)		

৫.২ Precision, Recall এর ভেন ডায়াগ্রাম

← মডেল "Positive" বলেছে →

| সব Predicted Positive |

| _____ |

| TP (সত্যিকার Positive) | ← Precision = TP / এই বক্সের মোট

| _____ |

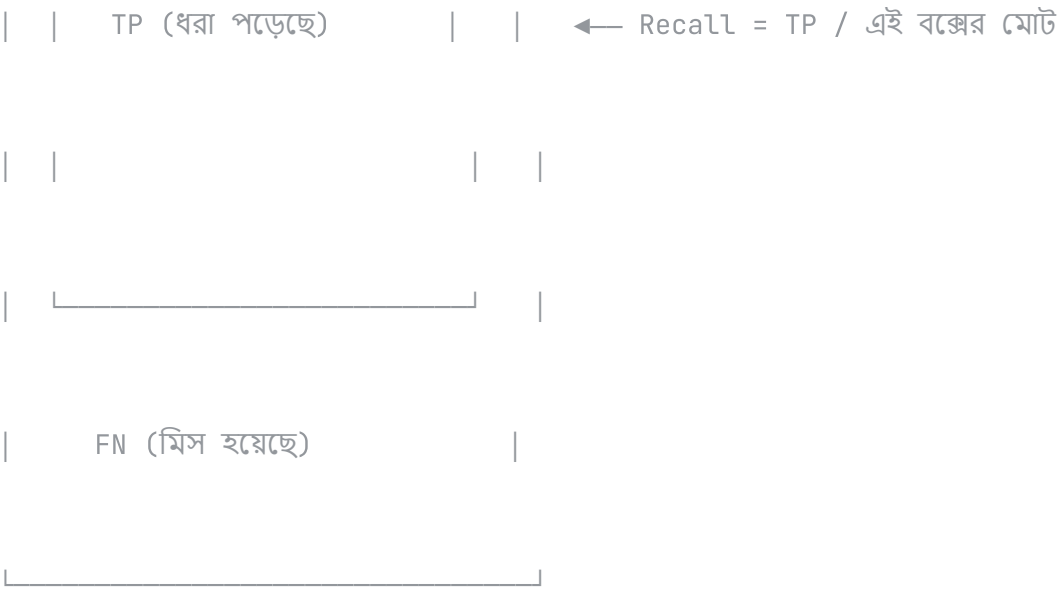
| _____ |

| FP (ভুল Positive) |

← সব আসল Positive →

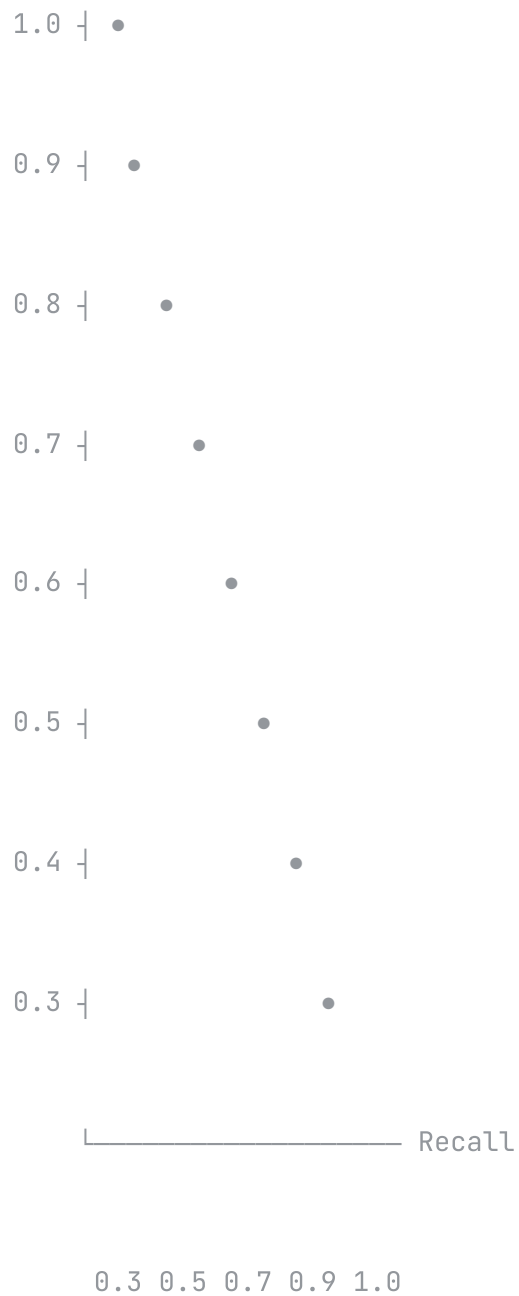
| সব Actual Positive |

| _____ |



৫.৩ Precision-Recall Trade-off curve

Precision



▲ Threshold কমালে → Recall বাড়ে, Precision কমে

▼ Threshold বাড়ালে → Precision বাড়ে, Recall কমে

৫.৪ কোন Metric কখন ব্যবহার করব?

তোমার সমস্যা কী?



| FP বেশি ক্ষতিকর? |

| (Spam Filter, | — হ্যাঁ —→ PRECISION ব্যবহার করো

| Legal Systems) |

| না



| FN বেশি ক্ষতিকর?|

| (Cancer Test, | — হ্যাঁ —→ RECALL ব্যবহার করো

| Fraud, Disaster)|

| না



| Data Imbalanced? | — হ্যাঁ —→ F1-SCORE ব্যবহার করো

| উভয়ই গুরুত্বপূর্ণ? |

| না



ACCURACY ব্যবহার করো (Balanced Data)

৬. Real-world Use Cases

Use Case ১: Medical Diagnosis (ক্যান্সার সনাক্তকরণ)

সমস্যা: Mammogram পরীক্ষায় স্তন ক্যান্সার সনাক্তকরণ। **গুরুত্বপূর্ণ Metric: Recall** — একটি ক্যান্সার মিস করা মানে রোগী চিকিৎসা পাবে না। **প্রয়োগ:** Google Health-এর AI mammography model-এ Recall বাড়াতে threshold কমিয়ে রাখা হয়, যার ফলে FP বেশি হলেও FN কম থাকে। Radiologist পরে confirm করেন।

Use Case ২: 📧 Email Spam Detection

সমস্যা: Gmail-এর Spam Filter **গুরুত্বপূর্ণ Metric: Precision** — একটি গুরুত্বপূর্ণ ইমেইল Spam Folder-এ গেলে ব্যবসায়িক ক্ষতি। **প্রয়োগ:** Gmail, Outlook সবাই High Precision বজায় রাখে — কিছু Spam ইনবক্সে গেলেও চলবে, কিন্তু গুরুত্বপূর্ণ ইমেইল হারানো যাবে না।

Use Case ৩: 💳 Credit Card Fraud Detection

সমস্যা: প্রতি মিলিয়ন transaction-এ মাত্র কয়েকটি Fraud (মারাত্মকভাবে Imbalanced!) **গুরুত্বপূর্ণ Metric: F1-Score & Recall** — একটি Fraud মিস করা আর বেশি FP (blocked cards) উভয়ই ক্ষতিকর। **প্রয়োগ:** Visa, Mastercard, PayPal — সবাই F1-Score optimize করে এবং ব্যালেন্স করে।

Use Case 8: 🔍 COVID-19 Testing

পরিস্থিতি: PCR test কতটা নির্ভরযোগ্য?

- **High Recall চাই:** সব COVID+ রোগী সনাক্ত হোক (FN মানে Quarantine না করা)
- **Moderate Precision:** কিছু False Positive মানে unnecessary Quarantine — গ্রহণযোগ্য

Use Case ৫: 🚗 Self-Driving Car (পথচারী সনাক্তকরণ)

সমস্যা: রাস্তায় মানুষ আছে কিনা সনাক্ত করা। **গুরুত্বপূর্ণ Metric: Recall (লাইফ-ক্রিটিক্যাল!)** — একজন পথচারীও মিস করা যাবে না। **প্রয়োগ:** Tesla, Waymo সবার model-এই Recall প্রায় ৯.০ রাখা হয়।

৭. ⚖️ Pros & Cons

Confusion Matrix

সুবিধা অসুবিধা 

সম্পূর্ণ ছবি দেখায়

বড় multiclass-এ বোঝা কঠিন

প্রতিটি ধরনের ভুল আলাদা করে দেখায়

সরাসরি একটি সংখ্যায় বলা যায় না

FP ও FN আলাদা করে analyze করা যায়

বড় dataset-এ visualization জটিল হয়

Precision

সুবিধা অসুবিধা 

False Alarm কমায়

FN উপেক্ষা করে

সহজ ব্যাখ্যাযোগ্য

একা ব্যবহারে অসম্পূর্ণ চিত্র দেয়

Spam Filter, Legal case-এ আদর্শ

Recall-এর সাথে trade-off সামলাতে হয়

Recall

সুবিধা অসুবিধা 

কোনো Positive মিস না করার নিশ্চয়তা

FP উপেক্ষা করে

Medical, Safety-critical context-এ আদর্শ

Recall ১.০ করলে সব কিছুকে Positive বলা হবে

Life-critical application-এ অপরিহার্য

Precision-এর সাথে trade-off

F1-Score

সুবিধা 

Precision ও Recall-এর ভারসাম্য

অসুবিধা 

Imbalanced data-তে এখনো বিভ্রান্তিকর হতে পারে

Imbalanced dataset-এ Accuracy চেয়ে ভালো

ব্যাখ্যা করা একটু কঠিন

Single metric দিয়ে comparison সহজ

উভয় class equally important ধরে নেয়

৮. Common Mistakes & Gotchas

ভুল ১: শুধু Accuracy দিয়ে সিদ্ধান্ত নেওয়া

❌ ভুল পদ্ধতি – Imbalanced data-তে Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Model is {accuracy*100:.1f}% accurate!") # 95% হলেই Model ভালো নয়!
```

✅ সঠিক পদ্ধতি

```
print(classification_report(y_test, y_pred)) # সব metric দেখো
```

বাস্তব বিপদ: ৯% Disease data-তে সবাইকে "Healthy" বললে 99% accurate কিন্তু 0% useful

ভুল ২: Test Data-তে Threshold না দেখে সিদ্ধান্ত নেওয়া

❌ ভুল – Default threshold 0.5 সবসময় সেরা

```
y_pred = model.predict(X_test) # 0.5 threshold ব্যবহার করছে
```

✅ সঠিক – Threshold tune করো

```
y_proba = model.predict_proba(X_test)[: , 1] # Probability বের করো
```

```
threshold = 0.3 # Recall বাড়াতে threshold কমাও
```

```
y_pred_custom = (y_proba ≥ threshold).astype(int)
```

ভুল ৩: Macro F1 ব্যবহার করা যখন Weighted F1 দরকার

❌ ভুল – Imbalanced data-তে Macro F1 minor

```
f1_macro = f1_score(y_test, y_pred, average='macro')
```

✅ সঠিক – Imbalanced data-তে Weighted F1

```
f1_weighted = f1_score(y_test, y_pred, average='weighted')
```

ভুল 8: Training Data-তে Metrics দেখা

❌ ভুল – Training data-তে সবসময় ভালো দেখে

```
y_train_pred = model.predict(X_train)

print(f"Train F1: {f1_score(y_train, y_train_pred):.3f}") # Misleading!
```

✅ সঠিক – শুধু Test data-তে evaluate করো

```
y_test_pred = model.predict(X_test)

print(f"Test F1: {f1_score(y_test, y_test_pred):.3f}") # Real performance
```

ভুল ৫: FP ও FN-এর ব্যবহারিক মানে না বোঝা

ডেটা সায়েন্টিস্ট হিসেবে সবসময় জিজ্ঞেস করো:

- এই প্রজেক্টে FP বেশি ক্ষতিকর, নাকি FN?
- Stakeholder-এর কাছে ব্যাখ্যা করতে পারব?

৯. Related Topics

আগে যা জানা দরকার (Prerequisites)

- ✓ **Logistic Regression** — Binary classification-এর ভিত্তি
- ✓ **Probability & Statistics** — TP, FP বোঝার জন্য
- ✓ **Python & Scikit-learn** — Practical implementation
- ✓ **Imbalanced Dataset** — কেন Accuracy একা যথেষ্ট নয়?

পরে যা শেখা উচিত (Next Steps)

- 🔮 **ROC Curve & AUC-ROC** — Threshold নিরপেক্ষ মূল্যায়ন
- 🔮 **Precision-Recall Curve** — Imbalanced data-তে আরো ভালো visualization
- 🔮 **Cross-Validation** — একটি test set-এর উপর নির্ভর না করে
- 🔮 **Hyperparameter Tuning** — F1-Score maximize করতে Grid Search / Optuna
- 🔮 **SMOTE (Oversampling)** — Imbalanced dataset ঠিক করার পদ্ধতি
- 🔮 **Cost-Sensitive Learning** — FP ও FN-এ আলাদা ক্ষতি নির্ধারণ

অন্যান্য Evaluation Metrics

Metric	কখন দেখা
ROC-AUC	Threshold নির্বাচনে নিরপেক্ষ মূল্যায়ন
Log Loss	Probability calibration দেখতে
Matthews Correlation Coeff (MCC)	অত্যন্ত Imbalanced data-তে
Cohen's Kappa	Random chance-এর উপরে পারফরম্যান্স

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল

TP, TN, FP, FN মনে রাখার নিয়ম:

FIRST WORD → মডেল কি সঠিক?

True = হ্যাঁ, মডেল সঠিক বলেছে

False = না, মডেল ভুল বলেছে

SECOND WORD → মডেল কী বলেছে?

Positive = মডেল "আছে" বলেছে (Alarm বাজিয়েছে)

Negative = মডেল "নেই" বলেছে (চুপ থেকেছে)

Precision vs Recall মনে রাখার Trick:

 **Precision** = তুমি কি **সঠিক** স্থানে তীর ছুঁড়েছ?

(তুমি যত তীর ছুঁড়েছ, তার মধ্যে কতটা লক্ষ্যে লেগেছে?)

 **Recall** = লক্ষ্যের সব বিন্দু কি তুমি **ধরতে** পেরেছ?

(সব লক্ষ্যের মধ্যে তুমি কতটাকে hit করতে পেরেছ?)

F1 মনে রাখার Trick:



"F1 হলো Precision ও Recall-এর সম্মান, কিন্তু দুজনেই দুর্বল হলে সম্মানও দুর্বল!"

১ লাইনে সারসংক্ষেপ



"Accuracy মিথ্যা বলে, Confusion Matrix সত্য বলে; Precision বলে 'কতটা সঠিক', Recall বলে 'কতটা ধরলাম', আর F1 উভয়কে এক সুতোয় বাঁধে।"

চিটশিট

প্রশ্ন	উত্তর
FP বেশি ক্ষতিকর?	Precision maximize করো
FN বেশি ক্ষতিকর?	Recall maximize করো
উভয়ই গুরুত্বপূর্ণ?	F1-Score maximize করো
Balanced dataset?	Accuracy চলতে পারে
Imbalanced dataset?	F1 / ROC-AUC ব্যবহার করো

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | 🖨️ AI-Assisted Bengali ML Notes পর্ব ১ — মেশিন লার্নিং কোর | টপিক ১১/২৫

📖 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | 🖨️ AI-assisted Bengali ML Notes | পর্ব ১ — টপিক ১১/২৫